

Reuniones que combinan



CHEF-INSPIRED FOOD & WINE PAIRING

- First Course -

CENTENNIAL ROSÉ Seared Scottish Salmon

*Spanish Saffron Beurre Blanc, Risotto, Sundried
Tomato Tapenade, Basil Oil*

- Second Course -

CARRIAGE HOUSE WHITE Pasta Primavera

*House Made Pasta, Crispy Country Ham,
Aruggula, Preserved Lemon, Roasted Heirloom
Tomato, Parmesan Cream*

- Third Course -

AMERICAN LEGEND Smoked Pork Belly

*Stone Ground Grits, Sorghum Mustard Glaze, Crushed
Peanuts, Black Eye Pea Relish*

- Fourth Course -

RACE DAY RED Steak Frites

*Filet Mignon, Shoestring Fries, Truffle Butter, Rosemary
Demi Glace*





JOINS en SQL:

Metodologías de relacionamiento entre Tablas



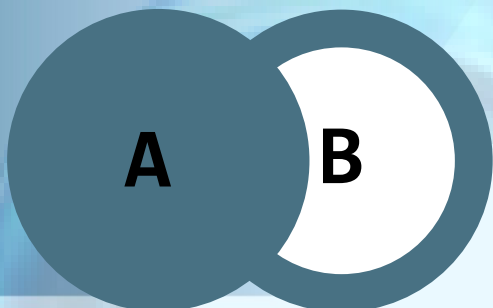
Objetivo: Dominar los diferentes tipos de joins (internos - INNER y externos - OUTER: left, right, full) para combinar datos de múltiples tablas y explorar los operadores de conjunto para manipulación avanzada de datos.

...descubriendo el poder del relacionamiento

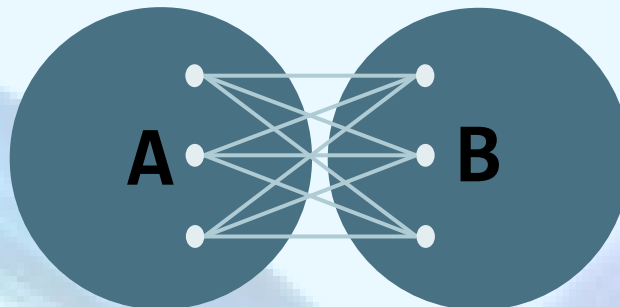


JOINS en SQL:

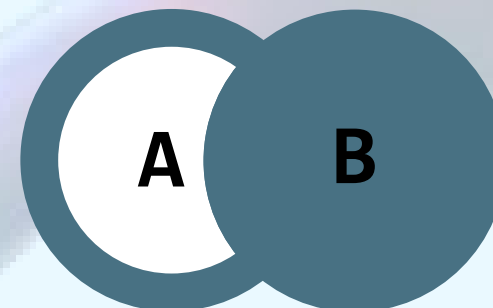
Metodologías de relacionamiento entre Tablas



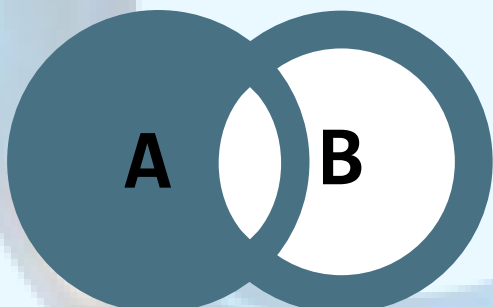
Left Join



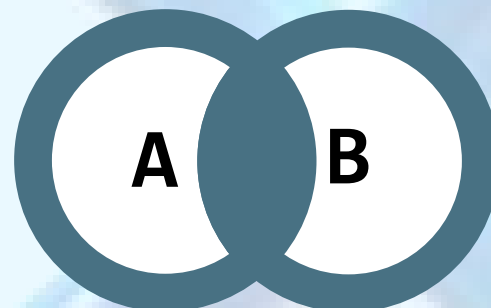
Cross Join



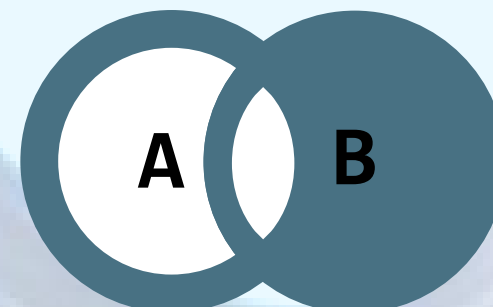
Right Join



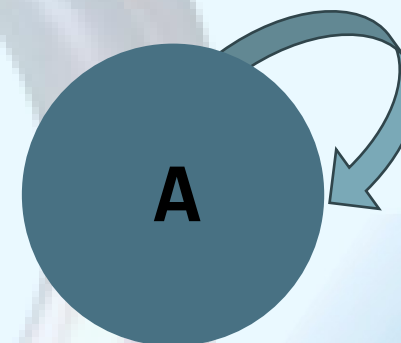
Left Anti Join
(is Null)



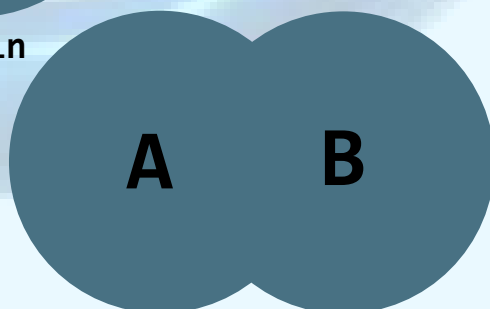
Inner Join



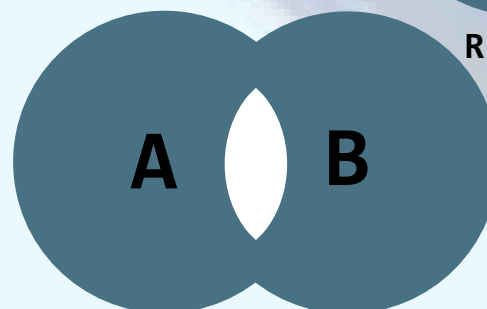
Right Anti Join
(is Null)



Self Join

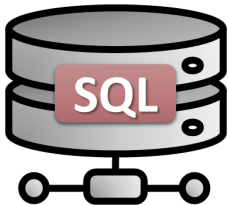


Full Outer Join

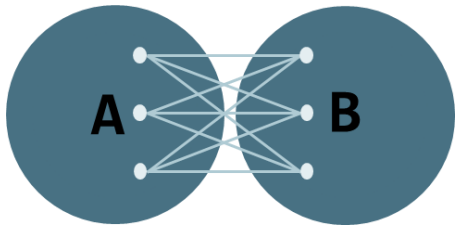


Full Anti Join (is Null)





CROSS JOIN – PRODUCTO CARTESIANO



SELECT *
FROM Empleados, Departamentos;

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



TABLA RESULTADO

ID_Emp	ID_Dpto	Nombre	ID_Dpto	Departamento
1	1	Aarón	1	Desarrollo
2	2	Adela	1	Desarrollo
3	3	Maria	1	Desarrollo
1	1	Aarón	2	Sistemas
2	2	Adela	2	Sistemas
3	3	Maria	2	Sistemas
1	1	Aarón	4	Finanzas
2	2	Adela	4	Finanzas
3	3	Maria	4	Finanzas

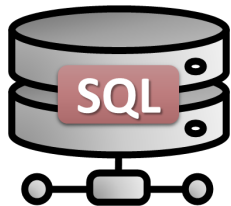
Uso implícito

```
SELECT *  
FROM Tabla1, Tabla2;
```

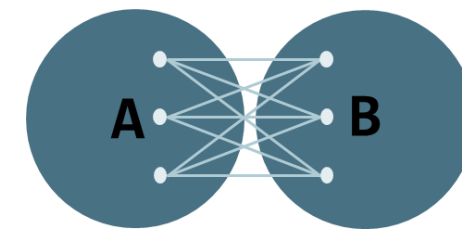
Uso explícito

```
SELECT *  
FROM Tabla1  
CROSS JOIN Tabla2;
```





CROSS JOIN – PRODUCTO CARTESIANO



```
SELECT *  
FROM Empleados, Departamentos  
WHERE empleados.ID_Dpto = departamentos.ID_Dpto;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

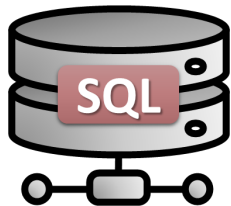
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



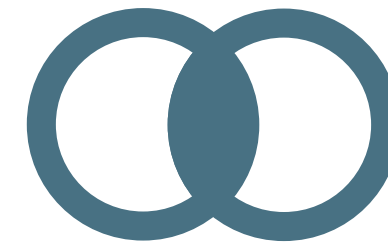
TABLA RESULTADO

ID_Emp	ID_Dpto	Nombre	ID_Dpto	Departamento
1	1	Aarón	1	Desarrollo
2	2	Adela	2	Sistemas

DML



INNER JOIN



```
SELECT e.Nombre, d.departamento  
FROM Empleados e  
INNER JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;
```

= JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas

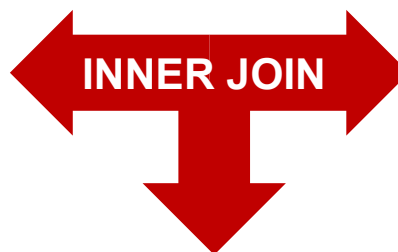
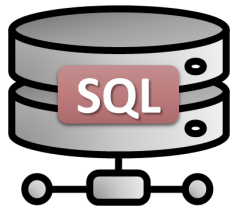


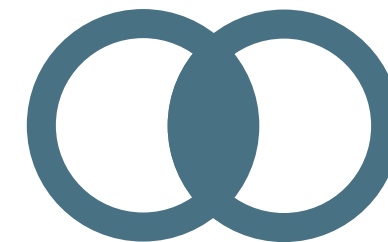
TABLA RESULTADO

Nombre	Departamento
Aarón	Desarrollo
Adela	Sistemas

DML

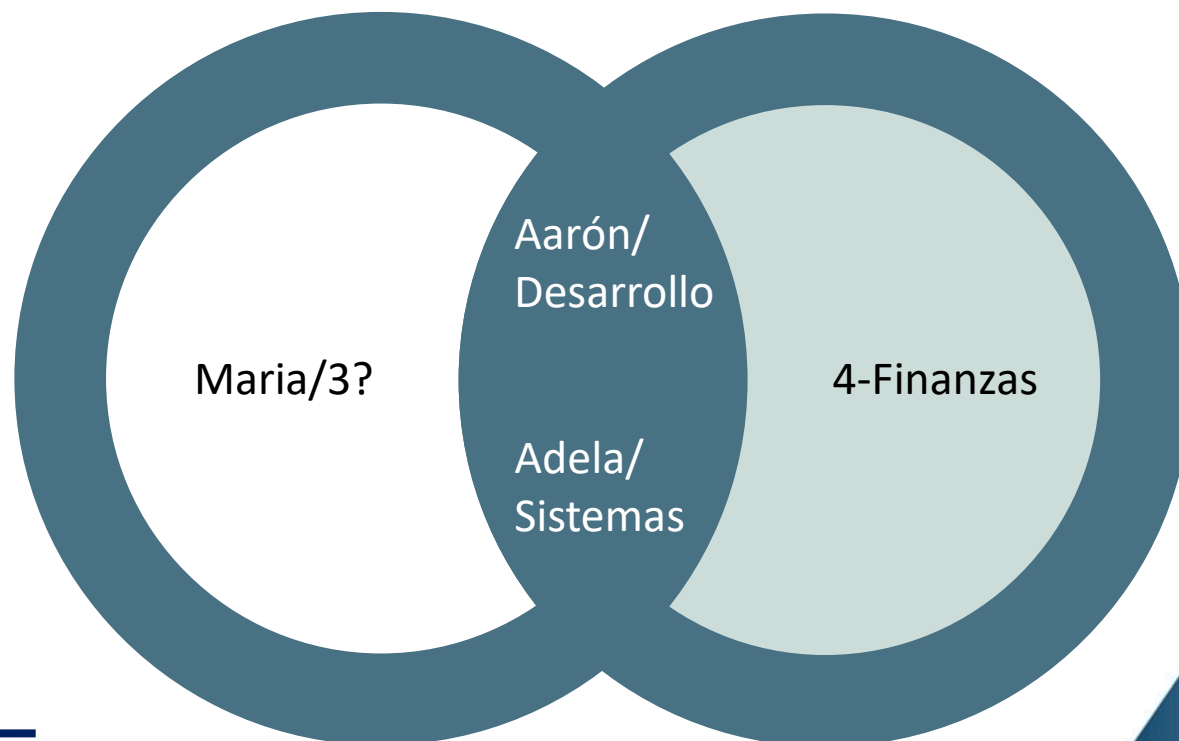


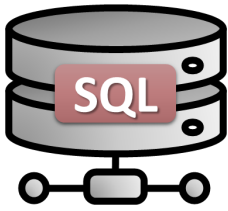
INNER JOIN



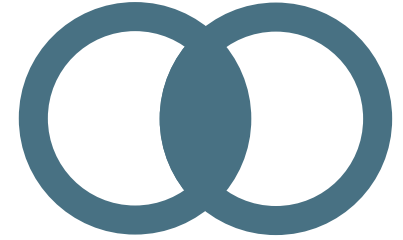
```
SELECT e.Nombre, d.departamento  
FROM Empleados e  
INNER JOIN Departamentos d ON e.ID_Dpto, d.ID_Dpto;
```

= JOIN Departamentos d ON e.ID_Dpto, d.ID_Dpto;





INNER JOIN



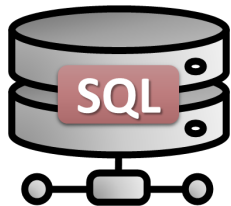
SELECT

```
Pedidos.ID_Pedido,  
Pedidos.FechaPedido,  
    Clientes.NombreCliente,  
    Productos.NombreProducto,  
    DetallePedidos.Cantidad,  
    DetallePedidos.Total,  
    Sucursales.NombreSucursal
```

FROM

```
    Pedidos  
INNER JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente  
INNER JOIN DetallePedidos ON Pedidos.ID_Pedido = DetallePedidos.ID_Pedido  
INNER JOIN Productos ON DetallePedidos.ID_Producto = Productos.ID_Producto  
INNER JOIN Sucursales ON Pedidos.ID_Sucursal = Sucursales.ID_Sucursal;
```

	ID_Pedido	FechaPedido	NombreCliente	NombreProducto	Cantidad	Total	NombreSucursal
1	101	2024-10-01	Juan	Televisor	2	40000.00	Sucursal A
2	101	2024-10-01	Juan	Cafetera	1	3000.00	Sucursal A
3	102	2024-10-02	Maria	Lavadora	1	15000.00	Sucursal B



LEFT JOIN



```
SELECT e.Nombre, d.departamento  
FROM Empleados e  
LEFT [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

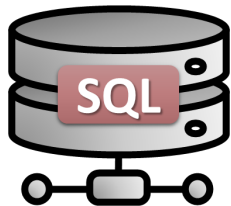
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



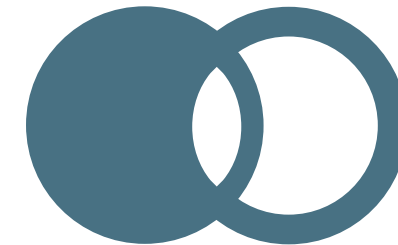
TABLA RESULTADO

Nombre	Departamento
Aarón	Desarrollo
Adela	Sistemas
Maria	NULL





LEFT ANTI JOIN: LEFT JOIN con NULL



```
SELECT e.Nombre, d.departamento
FROM Empleados e
LEFT [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto
WHERE d.ID_Dpto IS NULL;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

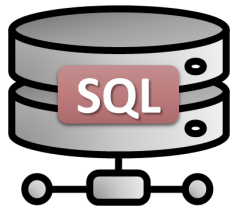
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



TABLA RESULTADO

Nombre	Departamento
Maria	NULL

DML



LEFT JOIN

SELECT

Pedidos.ID_Pedido,
Pedidos.FechaPedido,
Clientes.NombreCliente,
Pedidos.ID_Sucursal

FROM

Pedidos

LEFT JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente;

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	101	2024-10-01	Juan	1
2	102	2024-10-02	María	2
3	103	2024-10-03	NULL	3

LEFT ANTI JOIN

SELECT

Pedidos.ID_Pedido,
Pedidos.FechaPedido,
Clientes.NombreCliente,
Pedidos.ID_Sucursal

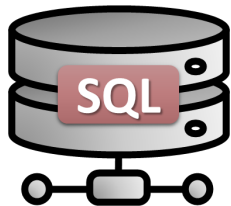
FROM

Pedidos

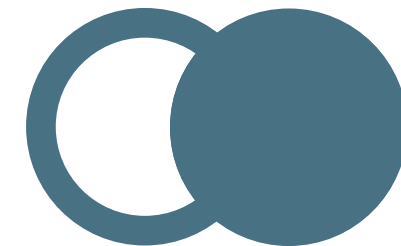
LEFT JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente

WHERE Clientes.ID_Cliente IS NULL;

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	103	2024-10-03	NULL	3



RIGHT JOIN



```
SELECT e.Nombre, d.departamento  
FROM Empleados e  
RIGHT [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

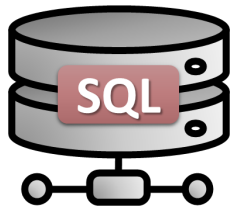
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



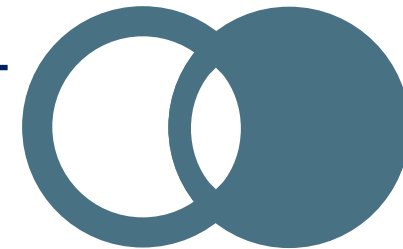
TABLA RESULTADO

Nombre	Departamento
Aarón	Desarrollo
Adela	Sistemas
NULL	Finanzas

DML



RIGHT ANTI JOIN: RIGHT JOIN con NULL



```
SELECT e.Nombre, d.departamento
FROM Empleados e
RIGHT [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto
WHERE e.ID_Dpto IS NULL;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

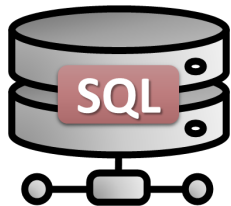
TABLA 2

ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



TABLA RESULTADO

Nombre	Departamento
NULL	Finanzas



RIGHT JOIN

SELECT

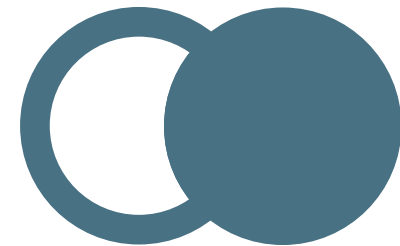
```
Pedidos.ID_Pedido,  
Pedidos.FechaPedido,  
Clientes.NombreCliente,  
Pedidos.ID_Sucursal
```

FROM

Pedidos

RIGHT JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente;

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	101	2024-10-01	Juan	1
2	102	2024-10-02	María	2
3	NULL	NULL	Pedro	NULL
4	NULL	NULL	Lucía	NULL
5	NULL	NULL	Ana	NULL



RIGHT ANTI JOIN

SELECT

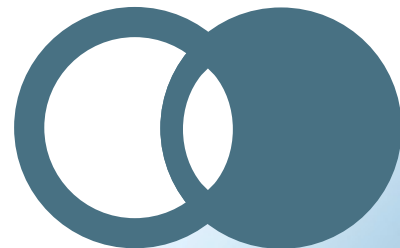
```
Pedidos.ID_Pedido,  
Pedidos.FechaPedido,  
Clientes.NombreCliente,  
Pedidos.ID_Sucursal
```

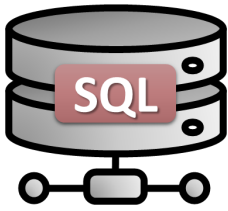
FROM

Pedidos

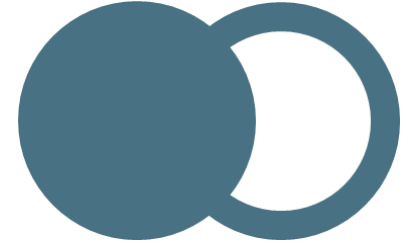
RIGHT JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente
WHERE Pedidos.ID_Pedido IS NULL;

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	NULL	NULL	Pedro	NULL
2	NULL	NULL	Lucía	NULL
3	NULL	NULL	Ana	NULL





LEFT JOIN – RIGHT JOIN



```
SELECT e.Nombre, d.departamento, e2.Nombre  
FROM Empleados e  
LEFT JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto  
RIGHT JOIN Empleados e2 ON d.ID_Dpto = e2.ID_Dpto;
```

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria



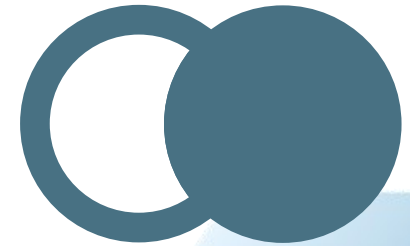
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas

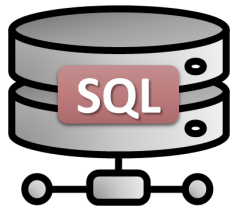
e.Nombre	d.ID_Dpto
Aarón	1
Adela	2
Maria	NULL



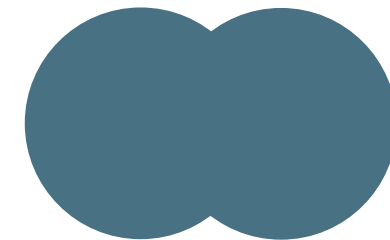
ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

e.Nombre	d.Departamento	e2.Nombre
Aarón	Desarrollo	Aarón
Adela	Sistemas	Adela
NULL	NULL	Maria





FULL OUTER JOIN



```
SELECT e.Nombre, d.departamento  
FROM Empleados e  
FULL [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

TABLA 2

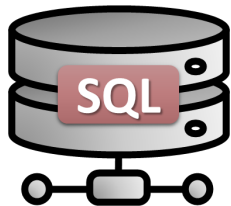
ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



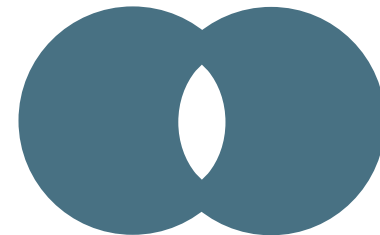
TABLA RESULTADO

Nombre	Departamento
Aarón	Desarrollo
Adela	Sistemas
Maria	NULL
NULL	Finanzas

DML



FULL ANTI JOIN: FULL JOIN con NULL



```
SELECT e.Nombre, d.departamento
FROM Empleados e
FULL [OUTER] JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto
WHERE e.ID_Dpto IS NULL or d.ID_Dpto IS NULL;
```

TABLA 1

ID_Emp	ID_Dpto	Nombre
1	1	Aarón
2	2	Adela
3	3	Maria

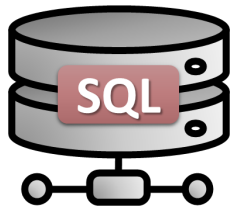
TABLA 2

ID_Dpto	Departamento
1	Desarrollo
2	Sistemas
4	Finanzas



TABLA RESULTADO

Nombre	Departamento
Maria	NULL
NULL	Finanzas



FULL JOIN

SELECT

```
Pedidos.ID_Pedido,  
Pedidos.FechaPedido,  
Clientes.NombreCliente,  
Pedidos.ID_Sucursal
```

FROM

```
Pedidos
```

```
FULL JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente;
```

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	101	2024-10-01	Juan	1
2	102	2024-10-02	Maria	2
3	103	2024-10-03	NULL	3
4	NULL	NULL	Pedro	NULL
5	NULL	NULL	Lucía	NULL
6	NULL	NULL	Ana	NULL

FULL ANTI JOIN

SELECT

```
Pedidos.ID_Pedido,  
Pedidos.FechaPedido,  
Clientes.NombreCliente,  
Pedidos.ID_Sucursal
```

FROM

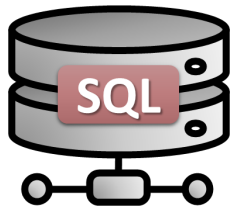
```
Pedidos
```

```
FULL JOIN Clientes ON Pedidos.ID_Cliente = Clientes.ID_Cliente
```

```
WHERE Pedidos.ID_Pedido IS NULL OR
```

```
Clientes.ID_Cliente IS NULL;
```

	ID_Pedido	FechaPedido	NombreCliente	ID_Sucursal
1	103	2024-10-03	NULL	3
2	NULL	NULL	Pedro	NULL
3	NULL	NULL	Lucía	NULL
4	NULL	NULL	Ana	NULL



θ - CONDICIONES DE LA CLAUSULA ON

- ✓ **Igualdad:** Para combinar filas donde una columna en ambas tablas tiene el mismo valor.

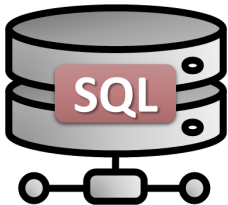
```
SELECT e.Nombre, d.Departamento  
FROM Empleados e  
INNER JOIN Departamentos d ON e.ID_Dpto = d.ID_Dpto;
```

- ✓ **Comparaciones:** Para combinar filas en función de criterios numéricos, de fecha, o cualquier tipo de comparación <, >, <=, >=.

```
SELECT p.NombreProducto, s.FechaStock  
FROM Productos p  
INNER JOIN Stock s ON p.PrecioUnitario > s.PrecioReferencia;
```

- ✓ **Condiciones múltiples (AND, OR):** Para combinar varias condiciones en la cláusula ON.

```
SELECT e.Nombre, d.Departamento  
FROM Empleados e  
INNER JOIN Departamentos d  
ON e.ID_Dpto = d.ID_Dpto AND e.Ciudad = d.Ciudad;
```



θ - CONDICIONES DE LA CLAUSULA ON

- ✓ **Uso de funciones:** Para transformar y comparar valores calculados.

```
SELECT e.Nombre, d.Departamento
FROM Empleados e
INNER JOIN Departamentos d
ON UPPER(e.Nombre) = UPPER(d.NombreJefe);
```

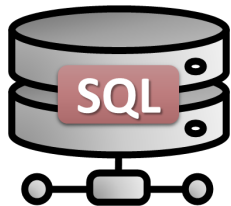
- ✓ **Subconsultas:** Para buscar dinámicamente valores en otras tablas.

- ✓ **Escalares
Recurrentes**

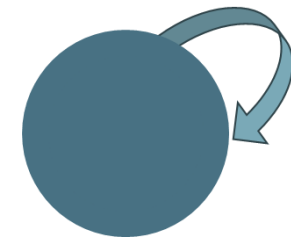
```
SELECT e.Nombre, d.Departamento
FROM Empleados e
LEFT JOIN Departamentos d
ON e.ID_Dpto = (SELECT MAX(ID_Dpto) FROM Departamentos);
```

- ✓ **Nulos (IS NULL, IS NOT NULL):** Para incluir o excluir filas basadas en la presencia de valores nulos.

```
SELECT e.Nombre, d.Departamento
FROM Empleados e
LEFT JOIN Departamentos d
ON e.ID_Dpto = d.ID_Dpto AND d.ID_Dpto IS NOT NULL;
```

SELF JOIN



```
SELECT e.Nombre AS Empleado,  
       g.Nombre AS Gerente  
FROM Empleados e  
LEFT JOIN Empleados g ON e.ID_Gte = g.ID_Emp;
```

TABLA 1

ID_Emp	ID_Gte	Nombre
1	NULL	Aarón
2	1	Adela
3	1	Maria

TABLA 2

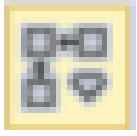
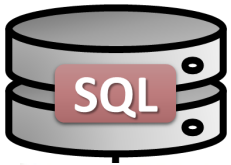
ID_Emp	ID_Gte	Nombre
1	NULL	Aarón
2	2	Adela
3	3	Maria



TABLA RESULTADO

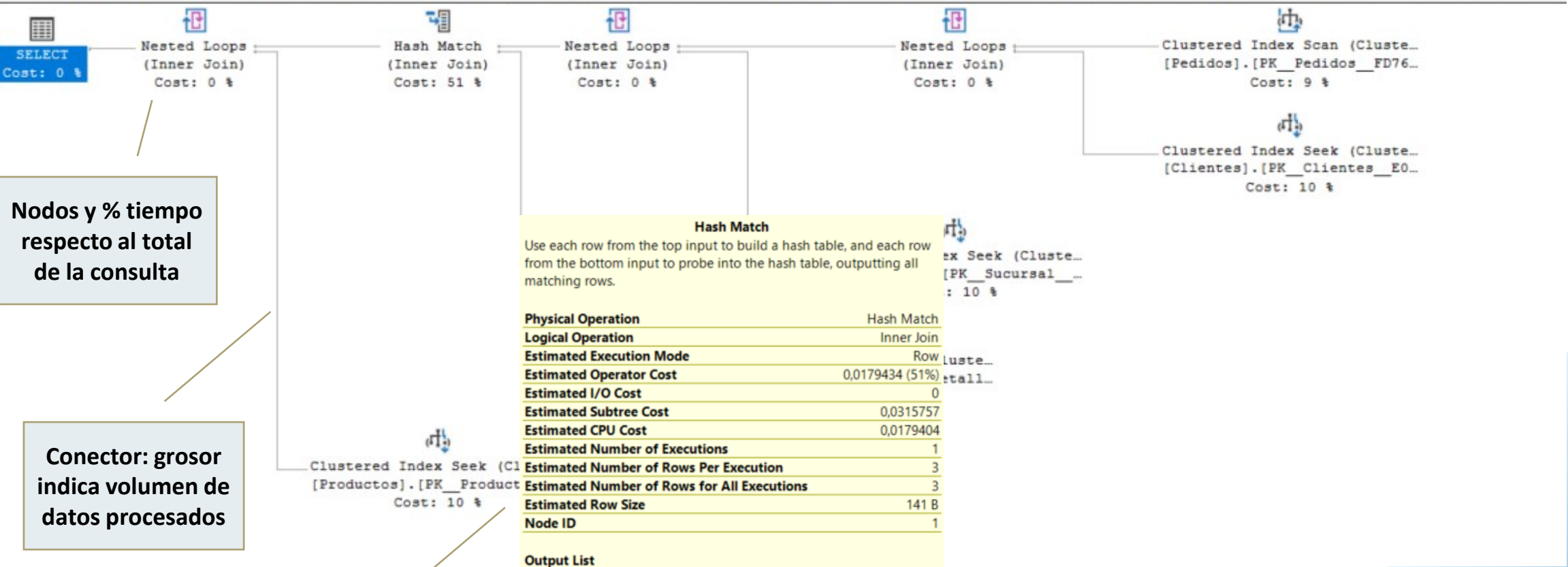
Empleado	Gerente
Aarón	NULL
Adela	Aarón
Maria	Aarón

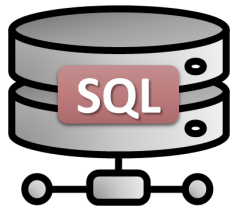
DML



OPTIMIZADOR – PLAN DE EJECUCIÓN

Query 24: Query cost (relative to the batch): 10%
; --- INNER JOIN --> Pedidos SELECT Pedidos.ID_Pedido, Pedidos.FechaPedido, Clientes.NombreCliente, Productos.NombreProducto, DetallePedidos.C





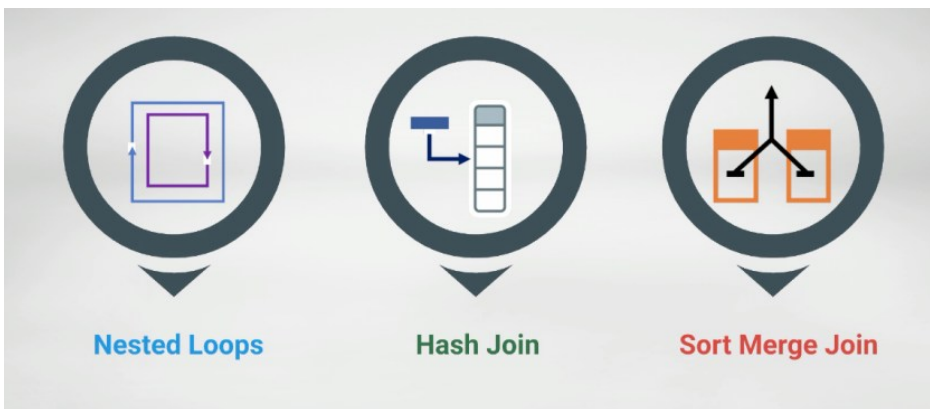
OPTIMIZADOR – PLAN DE EJECUCIÓN

HINTS

{ LOOP | HASH | MERGE }

HINTS o sugerencias son opciones o estrategias especificadas para que el procesador de consultas de SQL Server las aplique en instrucciones SELECT, INSERT, UPDATE o DELETE.

Las sugerencias reemplazan cualquier plan de ejecución que pueda seleccionar el optimizador de consultas para una consulta.



```
SELECT *  
FROM Productos p  
LEFT OUTER HASH JOIN Stock s  
ON p.ID_Producto = s.ID_Producto;
```

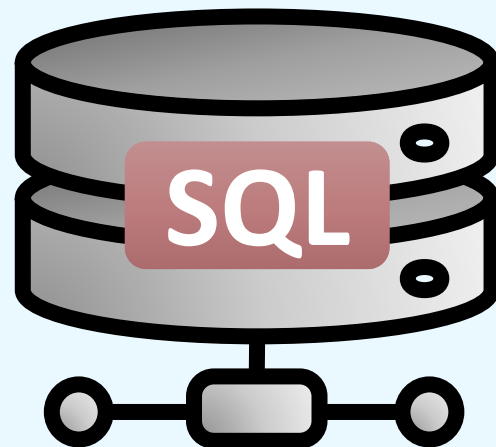
DML



JOINS en SQL:

Metodologías de relacionamiento entre Tablas

Los *joins* en SQL son la clave para desbloquear relaciones complejas entre tablas, permitiendo obtener información valiosa al combinar datos. Desde el análisis básico con *joins* internos hasta consultas avanzadas con *joins* externos, dominar su uso es esencial para cualquier administrador o desarrollador de bases de datos.



SQL - *Structured Query Language*



Bibliografía:

Introducción a los Sistemas de Bases de Datos - C. J. Date – 7º Edición

- **CAPÍTULO 11: Subconsultas y consultas correlacionadas**
- **CAPÍTULO 12: Operadores de conjunto**
- **CAPÍTULO 17: Seguridad y control de acceso**

Sistemas de Bases de Datos – Conceptos Fundamentales - Elmasri / Navathe – 5º Edición.

- **Capítulo 8: SQL-99: definición del esquema, restricciones, consultas y vistas.**
- **Capítulo 23: Seguridad en Bases de Datos**

Microsoft Docs – SQL Server: Join fundamentals

Próximo encuentro



Clase 9: SQL y Seguridad

Creación de procedimientos almacenados y funciones.

SQL dinámico.

Ejercitación práctica